

Albertitos Plan

Equipo Upistas

Pablo Sáez Morales

Antonio Ruiz Jiménez

Francisco Villalba Palomo

Javier Plazas Moyano

HackSpain 2026

1. Arquitectura

El problema

Alberto no necesita otro panel lleno de datos. Necesita saber qué facturas puede pagar, cuáles debe rechazar y cuáles requieren que una persona mire algo que el sistema no puede demostrar.

El reto consiste en procesar 540 facturas —500 iniciales y 40 del segundo lote— y cruzarlas con un maestro de proveedores, los pedidos y un ERP de 2009 lento e inestable.

Cada factura termina en uno de estos estados:

- **PAGAR:** los datos se han leído con seguridad y cumplen la norma.
- **NO_PAGAR:** existe un incumplimiento demostrado.
- **ESCALAR:** falta información o hay una duda que debe resolver Alberto.

La diferencia entre incumplimiento e incertidumbre es central. Si el sistema no puede leer un dato, no inventa la respuesta ni convierte la duda en una acusación.

La solución

La arquitectura separa tres responsabilidades:

1. **Leer** lo que contiene el documento.
2. **Comprobar** esa información contra fuentes fiables.
3. **Decidir** mediante reglas deterministas.

Cada factura se inspecciona y se busca primero una lectura compatible en caché. Si no existe, se intenta extraer el texto nativo. Solo cuando el documento está escaneado o el texto resulta insuficiente se utiliza OCR, si está disponible. La visión queda como último respaldo cuando el OCR tampoco permite leer los campos necesarios.

Si no hay un lector externo configurado o todos los lectores fallan, la factura se escala con el motivo y el resto del lote continúa.

PDF → caché o texto nativo → [si hace falta: OCR] → [si sigue siendo insuficiente: visión] → normalización → maestro + ERP → reglas → decisión → [si existe incertidumbre: revisión humana]

Esta separación permite utilizar modelos cuando aportan valor sin entregarles la decisión financiera. El OCR y la visión transcriben; nunca ordenan pagar.

Componentes y responsabilidades

La aplicación es un monolito modular. Dominio, web, persistencia e integraciones se despliegan juntos, pero mantienen límites explícitos.

Esto nos da la sencillez operativa de un único sistema sin mezclar las reglas con Django, Firecrawl o el ERP. El coste es mantener esos límites; por eso el repositorio comprueba automáticamente algunas dependencias prohibidas.

Componente	Qué hace	Por qué existe
Inspector PDF	Detecta texto, escaneos, adjuntos y contenido oculto	Un PDF puede contener más de lo que muestra el visor
Lector escalonado	Usa texto nativo y, cuando hace falta, Firecrawl o visión	Cada técnica tiene distinta fiabilidad, latencia y coste
Dominio	Aplica la norma y sus prioridades	La decisión debe ser reproducible y comprobable
Maestro	Conserva proveedores y pedidos versionados	Evita depender continuamente de Excel
Copia del ERP	Mantiene una fotografía local de sus datos	Reduce peticiones y evita decisiones incoherentes
DBOS y persistencia	Conservan lecturas, decisiones y ejecuciones	Permiten recuperar trabajo y explicar resultados
Web de Alberto	Presenta decisiones, evidencias y revisiones	La complejidad técnica no debe trasladarse al usuario
Asistente	Consulta información y propone acciones limitadas	Ayuda a trabajar sin adquirir autoridad sobre los pagos

Leer barato antes de leer caro

El sistema no envía todas las facturas a un modelo.

Primero intenta extraer el texto con PyMuPDF. Si la página está escaneada y el OCR está disponible, utiliza Firecrawl. Solo cuando el OCR sigue sin aportar campos suficientes recurre a visión.

Esta cascada tiene una razón práctica:

- El texto nativo es rápido, reproducible y no consume créditos.
- El OCR resuelve documentos escaneados.
- La visión cubre los casos difíciles, pero tiene más coste y variabilidad.

Cada lectura conserva el método, la versión, el tiempo, los errores y el consumo informado por el proveedor. Una lectura fallida no se guarda como si fuera correcta.

El presupuesto completo de lectura es de 300 segundos por documento. Firecrawl y la visión tienen 120 segundos de tiempo de red por defecto. Si una factura agota el tiempo, escala y el resto del lote continúa.

Cómo convertimos un fallo de Firecrawl en una mejora real

Las facturas escaneadas plantearon un problema que no aparecía en los PDF con texto.

Para aplicar OCR, cada página se renderizaba como una imagen PNG a 200 ppp. Firecrawl no acepta imágenes sueltas, así que esa imagen se introducía en un PDF de una página. PyMuPDF incrustaba el PNG prácticamente sin comprimir y algunas peticiones alcanzaban unos 11 MB.

Firecrawl cortaba o rechazaba la conexión antes de devolver el resultado. Repetir la misma petición no servía: el archivo seguía pesando lo mismo.

La solución fue cambiar la preparación del documento:

1. Renderizamos la página escaneada.
2. Eliminamos el canal alfa si existe.
3. La convertimos a JPEG con calidad del 80 %.
4. Insertamos ese JPEG en el PDF temporal.
5. Enviamos el documento comprimido a Firecrawl.

En el caso medido, el archivo pasó de unos 11 MB a alrededor de 60 KB y Firecrawl respondió en aproximadamente cinco segundos.

El beneficio es directo: menos rechazos, menos tiempo de subida y menos memoria. El trade-off es que JPEG tiene pérdida. Una compresión excesiva podría deteriorar números pequeños, por lo que el 80 % equilibra peso y legibilidad.

La recuperación distingue el tipo de error:

- Los fallos de transporte, 429 y 5xx se reintentan hasta tres veces con espera progresiva.
- Un **Retry-After** del proveedor se respeta.
- Los errores 4xx, las respuestas inválidas o la ausencia de texto no se repiten sin cambiar la causa.
- Si el OCR no puede completarse, se registra el error y la factura queda en **ESCALAR**.

Las llamadas también se serializan mediante un cerrojo compartido, porque el proveedor cortaba conexiones cuando recibía varias páginas al mismo tiempo.

No ocultamos el fallo: lo convertimos en una parte observable y recuperable del pipeline.

Un dato leído por OCR no siempre demuestra un incumplimiento

Un OCR puede confundir una B con un 8 o un 5 con un 6. Por eso, el sistema distingue entre un dato leído de forma nativa y uno interpretado desde una imagen.

Si un NIF obtenido por OCR no tiene una forma válida, se considera no leído. Si un IBAN, importe, pedido o IVA leído por OCR no coincide, la factura escala para que Alberto lo compruebe sobre el documento.

Esto no significa desconfiar de todo el OCR:

- Si todos los datos coinciden, la factura puede pagarse.
- Si el ERP demuestra que el pedido ya está pagado, continúa siendo **NO_PAGAR**.
- Si existe una nota manipuladora independiente de la lectura, como una petición de cambiar la cuenta, el rechazo puede mantenerse.

El beneficio es evitar falsos rechazos por un carácter mal reconocido. El coste es que algunos escaneos necesitan revisión humana.

Facturas internacionales

El segundo lote incorpora facturas en español, inglés, francés, italiano, alemán, portugués y catalán.

El lector reconoce etiquetas como INVOICE, FACTURE, RECHNUNG, Sous-total, Imponibile, VAT, TVA, MwSt. o GESAMT. También entiende NIF e IBAN extranjeros y fechas escritas con palabras.

Ser una factura extranjera no cambia la decisión. Lo relevante es si sus datos pueden comprobarse y si está expresada en euros o en otra moneda.

Para las divisas seguimos este criterio:

- El importe se conserva en su moneda original.
- Nunca se compara directamente con un pedido en euros.
- Se utiliza una tabla fija y versionada de tipos de referencia.
- Se muestra la conversión aproximada y si queda dentro del 0,5 % del pedido.
- Alberto debe autorizar siempre el pago en divisa.

La tabla incluye USD, GBP, CHF, JPY, BRL y MXN. No utilizamos cotizaciones en directo porque una misma factura podría obtener resultados diferentes en dos ejecuciones.

El tipo de referencia ayuda a Alberto, pero no decide por él. Ese es el trade-off: perdemos automatización completa a cambio de reproducibilidad y control financiero.

Una divisa distinta tampoco oculta otros problemas. Un IBAN incorrecto, un proveedor inexistente o un pedido ajeno pueden seguir justificando **NO_PAGAR**.

Reglas y prioridades

La norma v3 comprueba:

- NIF e IBAN.
- Pedido e importe.
- Divisa.
- IVA y coherencia fiscal.
- Fecha.
- Calidad de lectura.
- Notas y contenido oculto.
- Estado del ERP.
- Pagos y documentos aprobados anteriormente.
- Coherencia entre maestro y ERP.

Las prioridades expresan el criterio de negocio:

A igual prioridad gana el resultado más restrictivo:

NO_PAGAR > ESCALAR > PAGAR

Las prioridades evitan que el resultado dependa del orden accidental de ejecución. Su trade-off es que cambiar una prioridad puede afectar a muchas facturas; por eso la norma está versionada y

Prioridad	Significado
400	No se puede verificar
350	Incumplimiento demostrado
340	Factura en otra moneda
300	Revisión por notas, contenido oculto o duda fiscal
100	Pago previo
50	Advertencias y contradicciones de referencia

separada del código de lectura.

Documentos que intentan influir en la decisión

Una factura no es una fuente de instrucciones.

El sistema detecta notas visibles, texto blanco sobre blanco, texto en modo invisible, contenido cubierto y archivos incrustados. Esos elementos se conservan como evidencia, pero nunca pueden desactivar una regla.

Cuatro facturas contienen mensajes invisibles que solicitan pagar. También existen pies invisibles inofensivos, por lo que “hay texto oculto” es una señal de revisión, no una prueba automática de fraude.

El segundo lote añade texto dibujado letra a letra. En `e18_P001` aparece una anotación superpuesta con importes diez veces mayores que los impresos. Sin esta detección, la factura cumplía las reglas; con ella, queda en **ESCALAR** para que una persona determine qué importe es válido.

ERP y maestro

El ERP es lento, limita peticiones, caduca sesiones y puede devolver **ORA-00600**. Consultarlo para cada factura multiplicaría el coste y permitiría que dos documentos del mismo lote se evaluarán con estados diferentes.

Por eso descargamos una copia completa y versionada.

El adaptador controla:

- Límites 429 y **Retry-After**.
- Errores **ORA-00600**.
- Caducidad de sesión.
- Descargas incompletas.
- Varias entradas para el mismo pedido.

La descarga medida de 516 asientos tarda alrededor de cuatro segundos, utiliza entre 30 y 31 peticiones y absorbe varios errores del ERP.

La web puede mantener una sincronización automática cada minuto. Si el ERP falla, se conserva la última copia correcta, se registra el intento y se vuelve a probar en el siguiente ciclo.

El maestro propio se alimenta mediante Excel o CSV. La importación ofrece vista previa, valida las filas y puede repetirse sin duplicar datos. Cada ejecución guarda las versiones exactas del maestro,

del ERP y de la norma.

Persistimos lo caro y recalculamos lo barato

La lectura de una factura puede requerir OCR o visión. Ejecutar las reglas sobre datos ya extraídos es mucho más barato.

La caché utiliza:

- El SHA-256 del documento.
- La versión del lector.

Si ambos coinciden, reutilizamos la lectura. La decisión se vuelve a calcular porque pueden haber cambiado la norma, el maestro, el ERP o las demás facturas del lote.

En una ejecución de 500 documentos:

- 479 lecturas salieron de caché.
- 21 necesitaron lectura.
- La lectura tardó 41,64 segundos.
- Las 500 decisiones se calcularon en 0,74 segundos.
- La evaluación de notas tardó 0,52 segundos.

No congelamos decisiones antiguas y tampoco pagamos dos veces el coste de leer el mismo documento.

El resultado final contiene una decisión única para cada una de las 540 facturas: 455 PAGAR, 43 NO_PAGAR y 42 ESCALAR.

Trazabilidad de una decisión real

La factura 2026-0233-A_catering.pdf muestra el recorrido completo.

Su SHA-256 es:

c045c6fbdaae8db9443bd53dd8755f84b9a36a8c645c6fd482ff70f10f564345

La lectura se realizó con pdf_unificado@8c79a388acf7de60a1e373c4. Tardó 8,012 segundos, utilizó extracción determinista y no necesitó OCR, visión ni tokens.

El inspector registró:

estructura reparada al abrir

La decisión utilizó:

- Norma v3.
- ERP b189d7434436.
- Maestro 5dba01fc0e4f.
- Pedido P0-2026-0492.

NIF, IBAN, importe, IVA, fecha, notas y ERP eran correctos. Sin embargo, `factura_41082.pdf` empleaba el mismo pedido.

Las facturas tenían números literales y conceptos diferentes. El sistema podía demostrar que compartían pedido, pero no cuál era el original ni si correspondían a operaciones distintas.

La comprobación `R5_duplicado` dejó esta evidencia:

Pedido duplicado PO-2026-0492 sin original inequívoco:
`2026-0233-A_catering.pdf, factura_41082.pdf`

El resultado fue **ESCALAR**.

Pagar automáticamente podría duplicar el pago. Rechazar automáticamente podría descartar una factura legítima. El sistema conserva la evidencia y entrega a Alberto exactamente la decisión que necesita tomar.

La ejecución terminó correctamente, pero la factura continúa sin revisión humana. Ese trabajo pendiente también forma parte de la trazabilidad.

Del resultado técnico a una decisión comprensible

La web no obliga a Alberto a interpretar JSON, hashes o nombres de reglas.

Puede consultar el documento original, el resultado y una explicación breve. También puede abrir una copia marcada:

- Los datos problemáticos aparecen rodeados en naranja.
- Los importes en divisa y el IVA extranjero pueden señalarse.
- Las notas ocultas se muestran.
- Si no existe una posición fiable —por ejemplo, en un escaneo— se añade una etiqueta en la parte superior.
- Una página final resume el resultado y lo que no se ha podido comprobar.

El original nunca se modifica.

La revisión humana se guarda junto a la decisión automática, no encima de ella. Así podemos reconstruir qué dijo el sistema y qué decidió después Alberto.

Asistente con límites reales

El asistente puede responder preguntas sobre facturas, proveedores, pedidos, ERP, importaciones y reglas.

También puede proponer acciones concretas, como marcar un pedido para revisión o añadir una nota. Esas acciones requieren confirmación, utilizan un token firmado de un solo uso y quedan auditadas.

No puede:

- Pagar o rechazar una factura.
- Modificar el ERP.

- Crear o borrar proveedores.
- Cargar o reprocesar lotes.
- Ejecutar una acción sin confirmación.

El asistente ayuda a navegar el sistema, pero no hereda autoridad financiera.

Escalabilidad y coste

El cuello de botella principal no es calcular las reglas. Es esperar al OCR, la visión, el ERP y el disco.

La arquitectura reduce ese coste:

- Texto nativo antes que OCR.
- OCR solo para los documentos que lo necesitan.
- Visión únicamente cuando el OCR es insuficiente.
- JPEG comprimido antes de enviar escaneos.
- Caché por contenido y versión.
- Una descarga completa del ERP.
- Reglas locales y deterministas.
- Tipos de cambio locales.
- Procesamiento independiente por documento.

SQLite cubre el uso local. PostgreSQL permite aumentar procesos sin cambiar el dominio. Si la carga crece, podemos añadir trabajadores antes de asumir el coste operativo de dividir la aplicación en microservicios.

Los límites actuales son claros:

- La regla fiscal internacional completa sigue pendiente.
- Los tipos de cambio son referencias, no cotizaciones del día.
- JPEG introduce compresión con pérdida.
- El vocabulario multilingüe no es un traductor universal.
- La barra de progreso web puede perderse al reiniciar, aunque el trabajo durable continúe.
- La clave compartida no ofrece usuarios ni roles.
- La capacidad distribuida todavía no se ha medido en producción.

2. ADRs y trade-offs

ADR 1. Pipeline de lectura escalonado, separado del motor de decisión

Contexto. Las facturas pueden contener texto nativo, imágenes escaneadas, estructuras dañadas, texto oculto o diseños difíciles de interpretar. Ninguna técnica de lectura resuelve todos los casos con el mismo coste y fiabilidad.

Además, interpretar un documento y decidir si debe pagarse son problemas diferentes. La lectura puede ser probabilística; la aplicación de una norma debe poder reproducirse.

Alternativas consideradas.

1. Utilizar únicamente extracción nativa.
2. Enviar todos los documentos directamente a OCR.
3. Enviar todos los documentos a un modelo visual.
4. Permitir que un modelo produzca directamente la decisión final.
5. Combinar varios lectores y aplicar después reglas deterministas.

Decisión. Aplicamos una cascada formada por inspección, extracción nativa, OCR y modelo visual. Cada nivel se utiliza solo cuando el anterior no aporta información suficiente.

El resultado de esa lectura se entrega a un motor de reglas determinista y versionado. Los modelos ayudan a interpretar el documento, pero no deciden directamente si se paga.

Por qué. La extracción nativa es rápida, barata y reproducible, pero no sirve para todos los escaneos. El OCR amplía la cobertura, aunque introduce coste y posibles errores. El modelo visual puede resolver casos difíciles, pero es más caro y menos predecible.

Separar lectura y decisión permite aprovechar cada tecnología sin entregar a un modelo generativo la responsabilidad final.

Beneficios.

- Reduce llamadas externas y consumo de tokens.
- Reserva las técnicas más caras para los documentos difíciles.
- Permite repetir una decisión a partir de la misma lectura.
- Hace posible cambiar las reglas sin cambiar el lector.
- Conserva evidencias independientes de la conclusión.
- Impide que una respuesta libre del modelo se convierta directamente en una orden de pago.

Trade-offs.

- El pipeline tiene más estados y modos de fallo.

- Hay que definir cuándo una lectura es suficiente.
- Distintos lectores pueden producir estructuras diferentes.
- Deben conservarse las versiones y el método utilizado.
- Un error de extracción puede propagarse a reglas deterministas si no se detecta.

Evidencia. La factura 2026-0233-A_catering.pdf se resolvió mediante extracción determinista, sin OCR, tokens ni modelo visual. La decisión posterior quedó vinculada a pdf_unificado@8c79a388acf7de60a1e373c4 y a la norma v3.

ADR 2. Persistir la lectura y recalcular las decisiones con reglas versionadas

Contexto. Leer un documento puede requerir OCR o modelos externos. En cambio, evaluar reglas sobre una lectura ya disponible es comparativamente barato.

Las normas, el maestro, el ERP y el contexto del lote pueden cambiar aunque el PDF permanezca idéntico.

Alternativas consideradas.

1. Volver a leer todos los PDF en cada ejecución.
2. Guardar únicamente la decisión final.
3. Reutilizar indefinidamente tanto la lectura como la decisión.
4. Persistir la lectura y recalcular la decisión con las versiones actuales.

Decisión. Persistimos la lectura por SHA-256 y versión del lector. Si el contenido y el lector son compatibles, reutilizamos esa lectura.

Las decisiones se recalculan en cada ejecución y registran la norma, el ERP, el maestro y las evidencias utilizadas.

Por qué. Guardar solo el resultado impediría explicar cómo se obtuvo. Reutilizar una decisión antigua ignoraría cambios en las reglas o los datos externos. Volver a leer todo elevaría coste y latencia sin aportar información nueva.

Beneficios.

- Reduce el coste de OCR y modelos.
- Permite probar una norma nueva sobre lecturas existentes.
- Facilita comparar resultados entre versiones.
- Conserva la procedencia de cada dato.
- Permite recuperar trabajo después de interrupciones.
- Hace que el coste de iterar sobre las reglas sea bajo.

Trade-offs.

- La caché necesita una política de invalidación fiable.
- Un cambio incompatible del lector obliga a generar nuevas lecturas.
- La base de datos conserva más información.
- Es necesario diferenciar lectura, decisión y revisión humana.
- Un error cacheado podría reutilizarse hasta invalidarlo.

Evidencia. En una ejecución de 500 documentos se reutilizaron 479 lecturas y solo 21 necesitaron procesarse. Las 500 decisiones se recalcularon en 0,74 segundos.

ADR 3. Trabajar con una copia local, completa y versionada del ERP

Contexto. Varias reglas necesitan consultar pedidos y estados del ERP. Consultarlo repetidamente introduce latencia, incoherencias temporales y dependencia de un servicio que puede fallar.

Alternativas consideradas.

1. Consultar el ERP para cada factura.
2. Consultarlo para cada regla.
3. Mantener una copia local sin versión.
4. Obtener una copia completa y versionada al comenzar el lote.
5. Replicar permanentemente todo el ERP.

Decisión. Realizamos una sincronización completa y trabajamos con una copia local identificada por su versión.

El ERP se utiliza en modo de lectura y sus errores transitorios se gestionan dentro del adaptador.

Por qué. Una factura procesada al principio y otra al final no deberían evaluarse contra estados diferentes. La copia proporciona una fotografía coherente y evita multiplicar peticiones.

Beneficios.

- Reduce consultas y latencia.
- Aísla las decisiones de fallos temporales.
- Permite reproducir una ejecución.
- Simplifica el procesamiento de lotes.
- Conserva la última copia válida.

- Hace visible cuándo se sincronizaron los datos.

Trade-offs.

- La copia puede quedar desactualizada.
- La sincronización inicial añade tiempo.
- Hay que comprobar que la descarga esté completa.
- Una versión local no representa necesariamente el estado actual.
- Los pedidos con varias entradas necesitan una política explícita.

Evidencia. La factura trazada utilizó la versión ERP b189d7434436. El adaptador gestiona ORA-00600, límites 429, caducidad de sesión y descargas incompletas.

ADR 4. Escalar la incertidumbre a revisión humana

Contexto. Algunas situaciones no pueden resolverse con seguridad. Un pedido repetido puede ser un duplicado, una corrección, un reenvío o dos facturas legítimas.

Obligar al sistema a responder siempre PAGAR o NO_PAGAR convertiría la falta de información en una decisión financiera.

Alternativas consideradas.

1. Pagar cuando no pueda demostrarse un incumplimiento.
2. Rechazar cualquier factura dudosa.
3. Elegir automáticamente el resultado más probable.
4. Introducir un estado explícito de revisión humana.

Decisión. Utilizamos PAGAR, NO_PAGAR y ESCALAR. Las ambigüedades materiales se entregan a Alberto junto con sus evidencias.

La decisión humana se registra sin eliminar la decisión automática.

Por qué. Un enfoque permisivo puede producir pagos indebidos. Uno excesivamente conservador puede bloquear facturas correctas. Escalar mantiene la seguridad sin fingir certeza.

Beneficios.

- Evita falsos pagos y falsos rechazos.
- Hace visible lo que no puede demostrarse.
- Conserva la responsabilidad humana.
- Permite aprender de los casos revisados.

- Mantiene evidencias y responsabilidades.
- Facilita priorizar el trabajo pendiente.

Trade-offs.

- La automatización no llega al cien por cien.
- Se crea una cola de revisión.
- El tiempo final depende de Alberto.
- La revisión necesita una interfaz propia.
- Demasiados escalados pueden convertirse en un cuello de botella.

Evidencia. 2026-0233-A_catering.pdf y factura_41082.pdf compartían pedido, pero tenían números y conceptos diferentes. El sistema produjo ESCALAR y conservó la revisión pendiente.

ADR 5. Organizar el sistema como un monolito modular con puertos y adaptadores

Contexto. El sistema reúne dominio, PDF, OCR, modelos, ERP, persistencia, web, importaciones y asistente. Estos componentes evolucionan a ritmos diferentes, pero no existe una necesidad demostrada de desplegarlos por separado.

Alternativas consideradas.

1. Construir una aplicación única sin límites internos.
2. Dividir desde el inicio el sistema en microservicios.
3. Mantener un monolito con módulos y contratos explícitos.
4. Implementar las integraciones directamente desde la web.

Decisión. Mantenemos un único despliegue, separando dominio, aplicación, adaptadores, infraestructura y web mediante puertos y contratos.

Las dependencias se orientan hacia el dominio y se comprueban automáticamente.

Por qué. Una aplicación acoplada sería sencilla al principio, pero dificultaría sustituir lectores y probar reglas. Los microservicios añadirían red, despliegues y consistencia distribuida sin resolver un problema actual.

Beneficios.

- Mantiene una operación sencilla.
- Permite probar el dominio sin servicios externos.
- Facilita sustituir adaptadores.

- Evita el coste prematuro de los microservicios.
- Conserva transacciones y trazabilidad.
- Permite extraer componentes si la carga futura lo exige.

Trade-offs.

- Los módulos comparten proceso y despliegue.
- Un fallo no aislado puede afectar al conjunto.
- La escalabilidad independiente es más limitada.
- Los límites deben protegerse activamente.
- Las excepciones pueden acumular deuda.
- Separar servicios más adelante seguirá teniendo un coste.

Evidencia. El repositorio separa dominio, aplicación, adaptadores, infraestructura y web. Sus contratos impiden que las vistas accedan directamente a lectores o integraciones que no les corresponden.